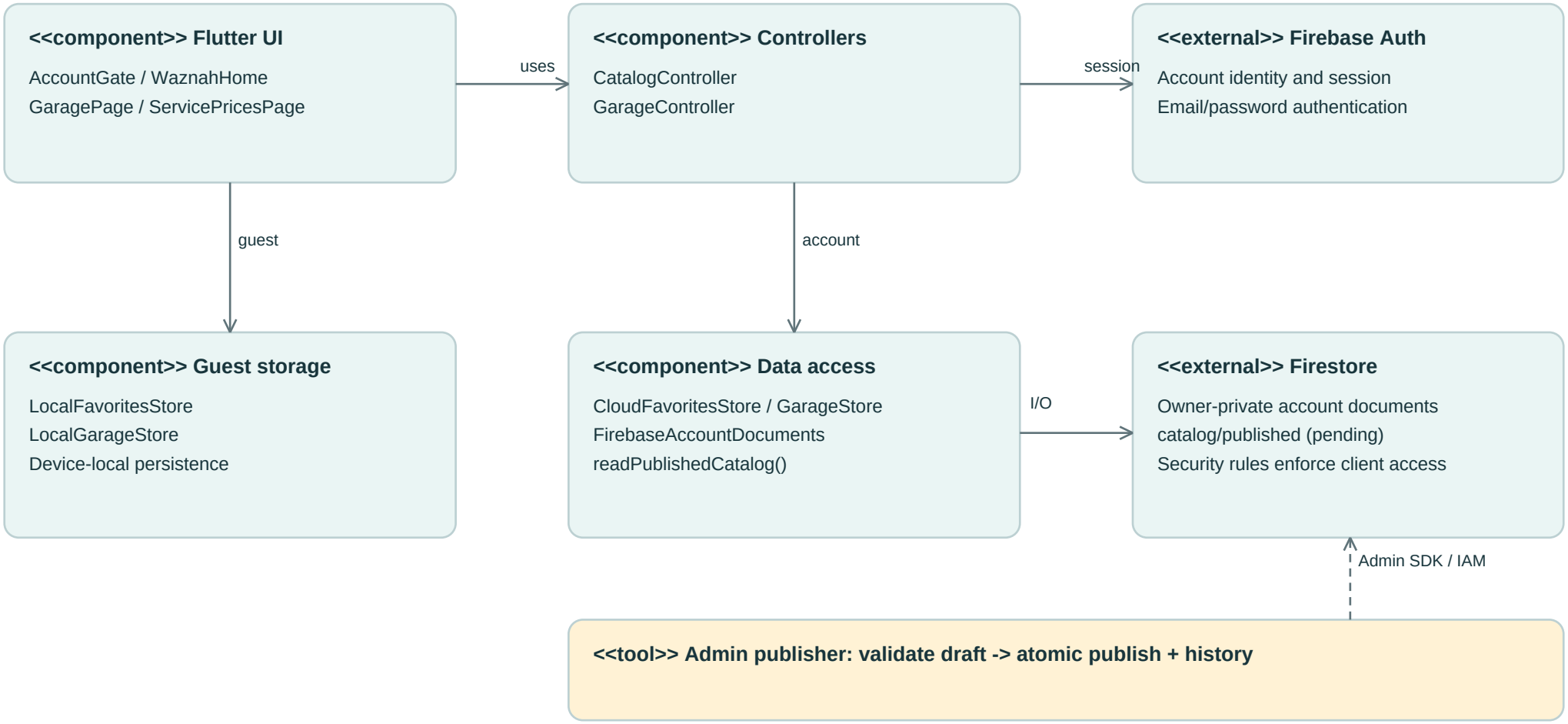
01 UML Model

Component diagram: presentation, application logic, storage and external systems.

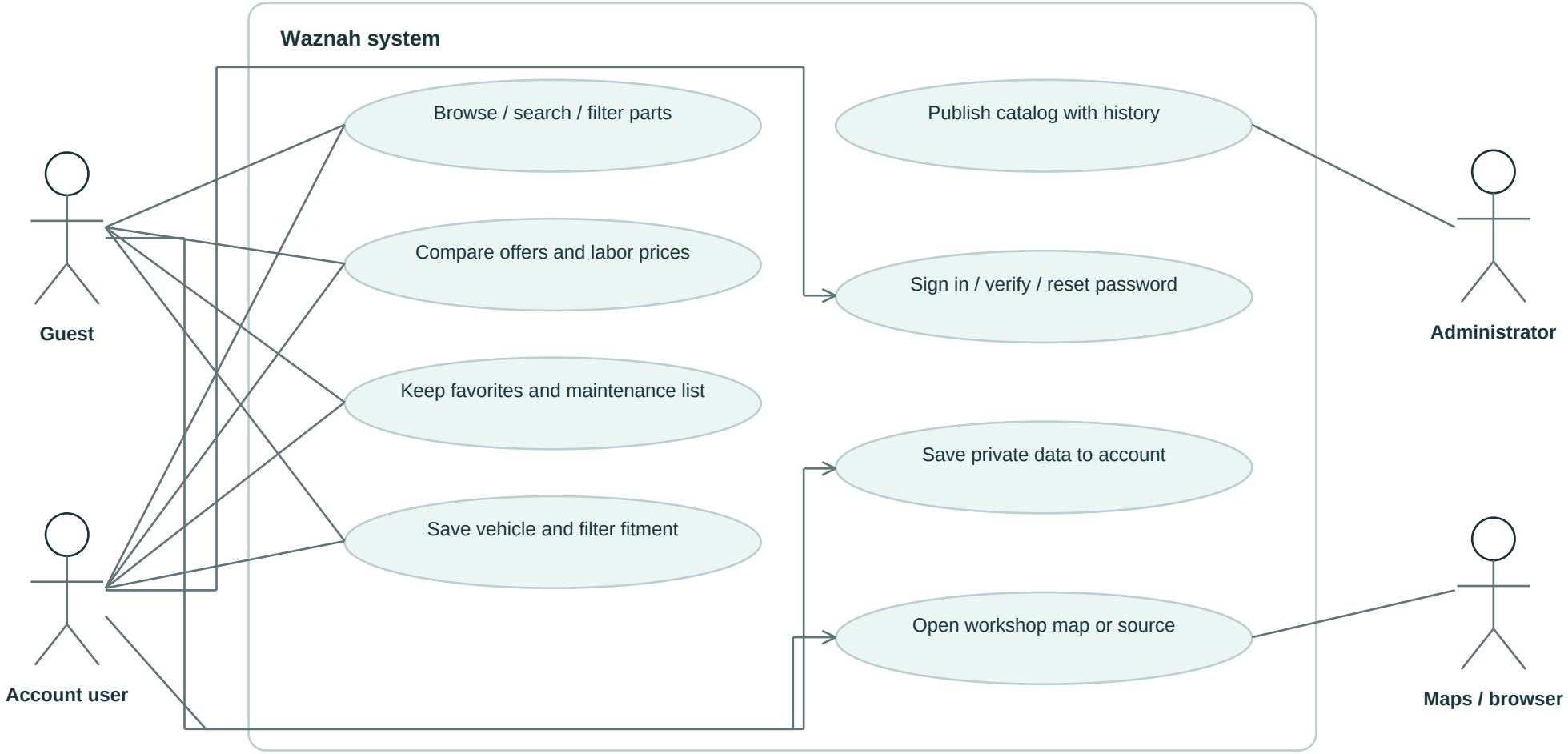


Solid arrows: runtime dependencies. Dashed arrow: administrator tooling, prepared but not activated for catalog publication. Client rules deny catalog writes; administrator access is separate.

Sources: lib/account_gate.dart, lib/main.dart, lib/catalog_repository.dart, lib/cloud_stores.dart, tool/admin/publish.mjs.

02 Use Case Diagram

Users explore prices and maintain lists; administrators maintain the source catalog.

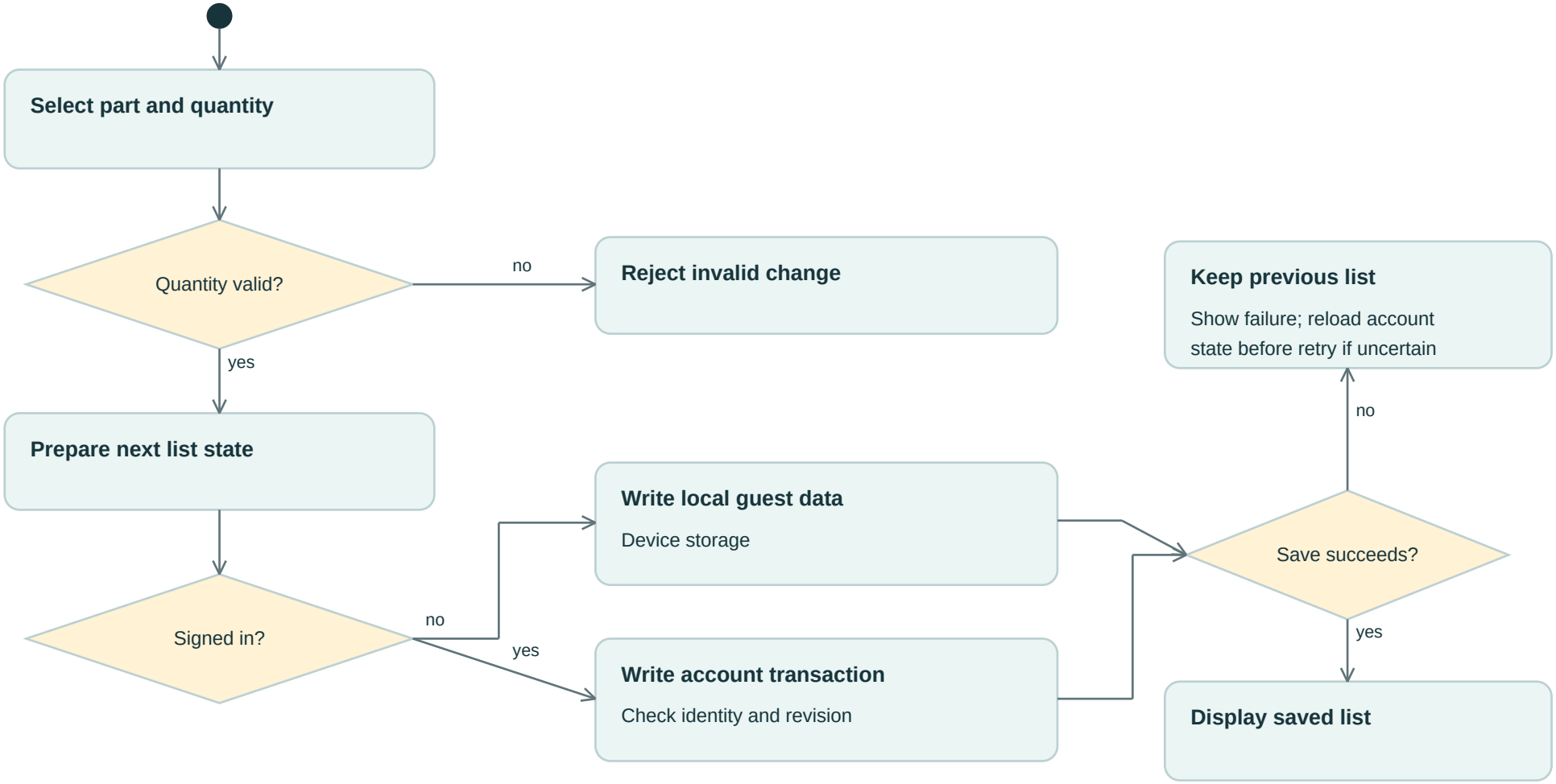


Guest lists are stored on the device; account lists use Firestore. Catalog publication is an administrator tool, not an app screen. Opening a map does not create a booking or track the user.

Sources: lib/main.dart, lib/account_gate.dart, lib/garage_page.dart, lib/service_prices_page.dart, tool/admin/publication.mjs.

03 Activity Diagram

Scenario: add a part or change its quantity in the maintenance list.

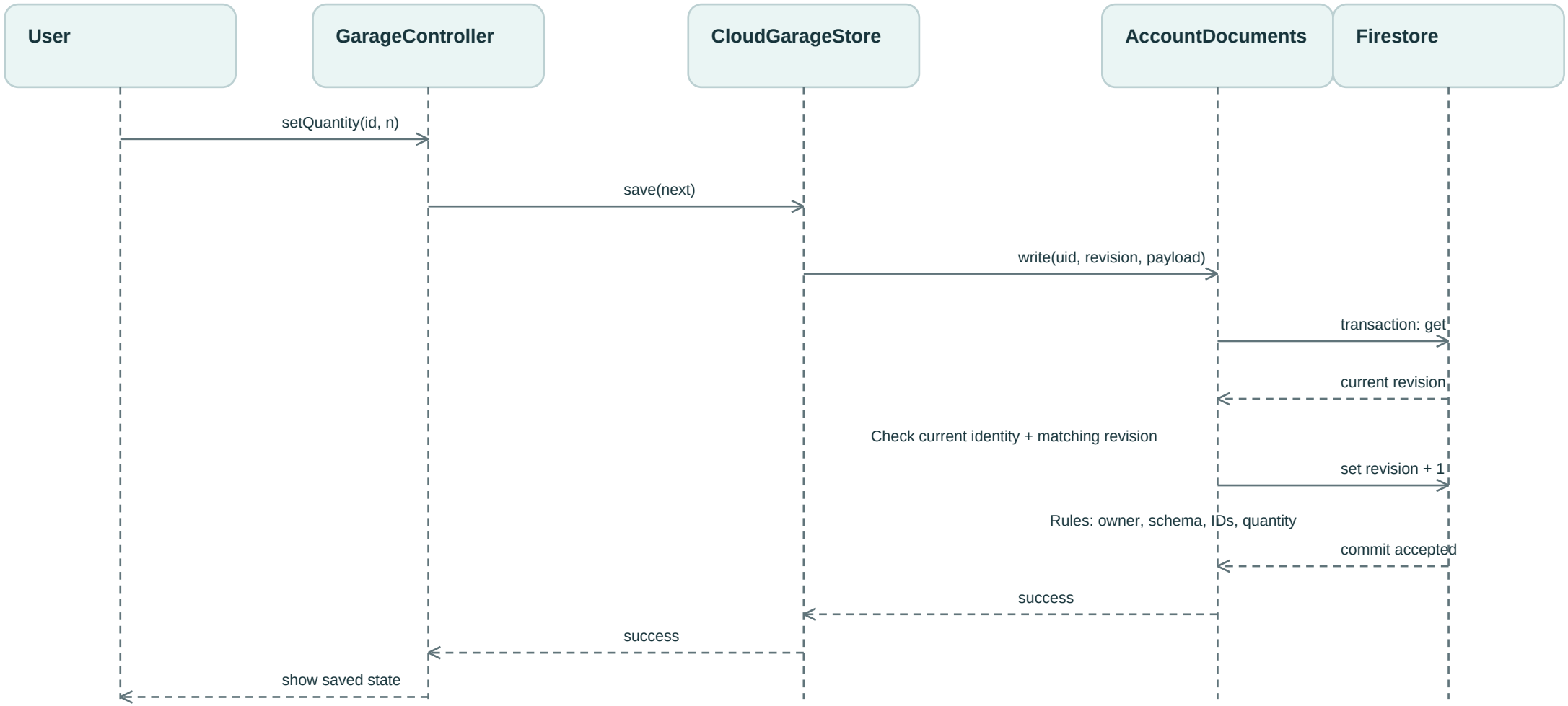


Quantities are whole numbers from 1 to 99. Removing an item is handled as a separate list change. The visible state is committed after a successful save; failed writes do not silently replace it.

Sources: lib/garage.dart (GarageController), lib/garage_page.dart, lib/cloud_stores.dart.

04 Sequence Diagram

Scenario: authenticated user saves a maintenance-list change.

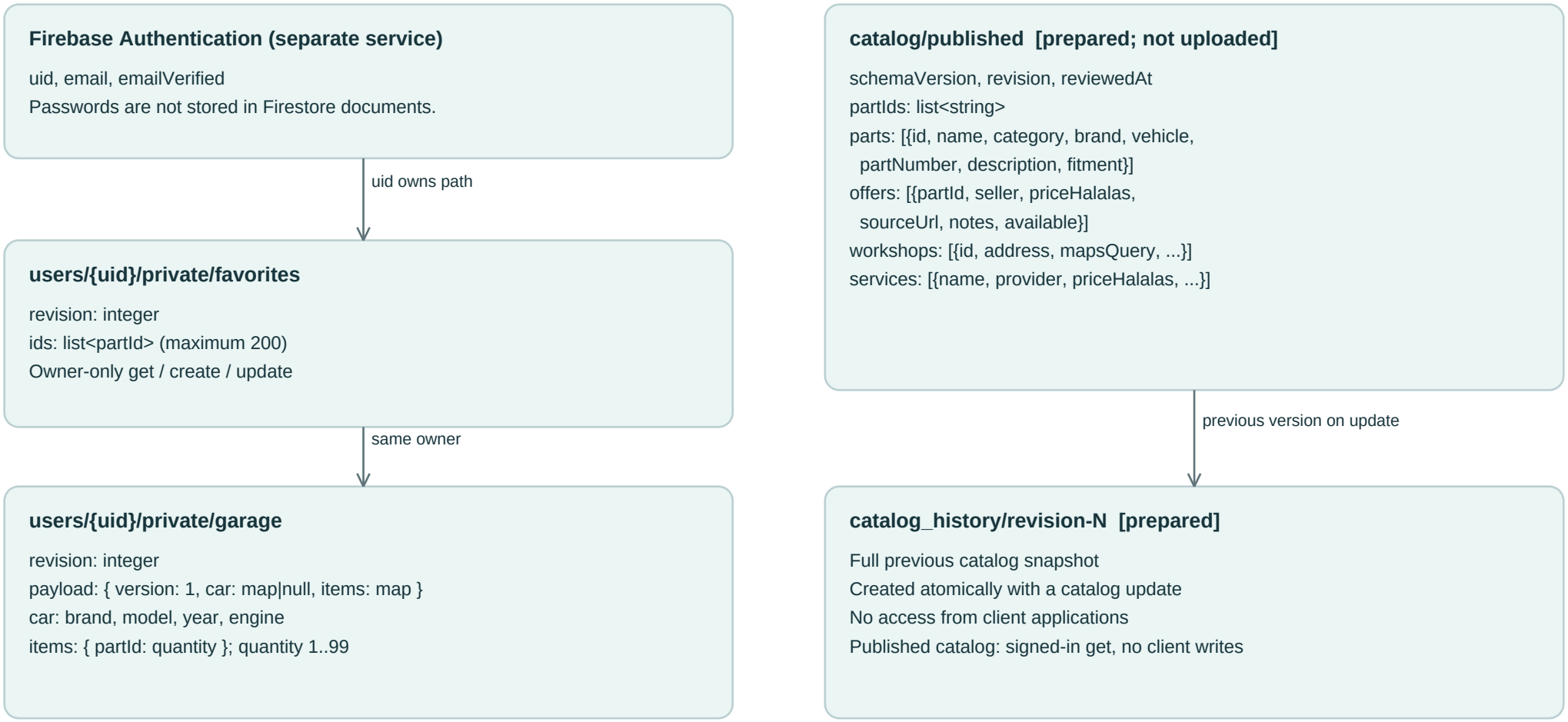


Failure branch: identity change, revision conflict or denied write throws an error. The UI keeps the previous state; the cloud store clears its cached revision after an uncertain write and requires reload.

Sources: lib/garage.dart, lib/cloud_stores.dart (FirebaseAccountDocuments.write), firestore.rules.

05 Database Diagram

Physical document layout: Cloud Firestore, not relational SQL tables.

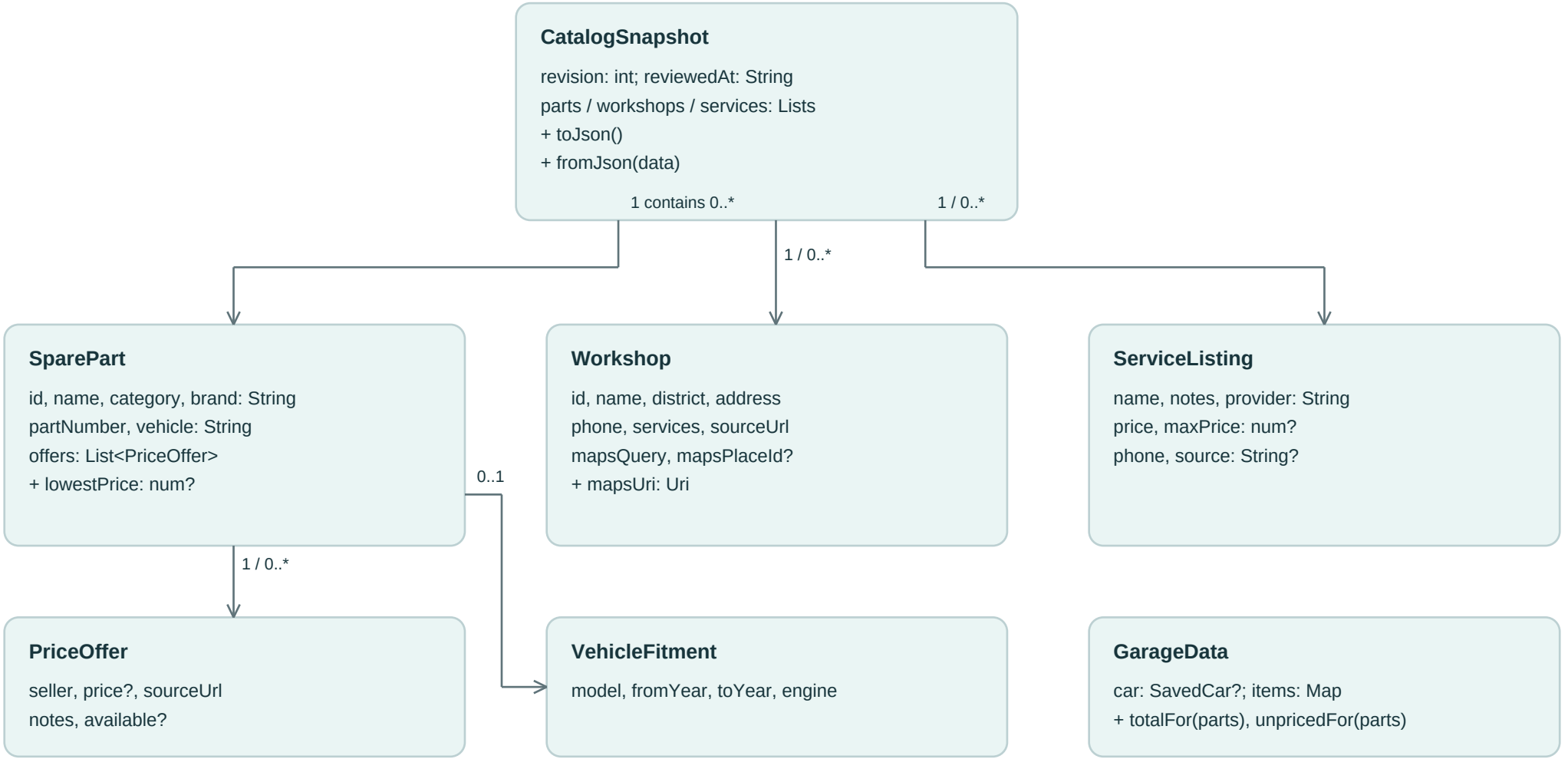


users/{uid} is a path namespace; a parent profile document is not required. References are stored as strings, not SQL foreign keys. Money is stored in integer halalas; missing prices remain null.

Sources: firestore.rules, lib/cloud_stores.dart, lib/catalog_snapshot.dart, tool/admin/publication.mjs.

06 Class Diagram

Core domain types and composition; UI widget classes omitted for readability.

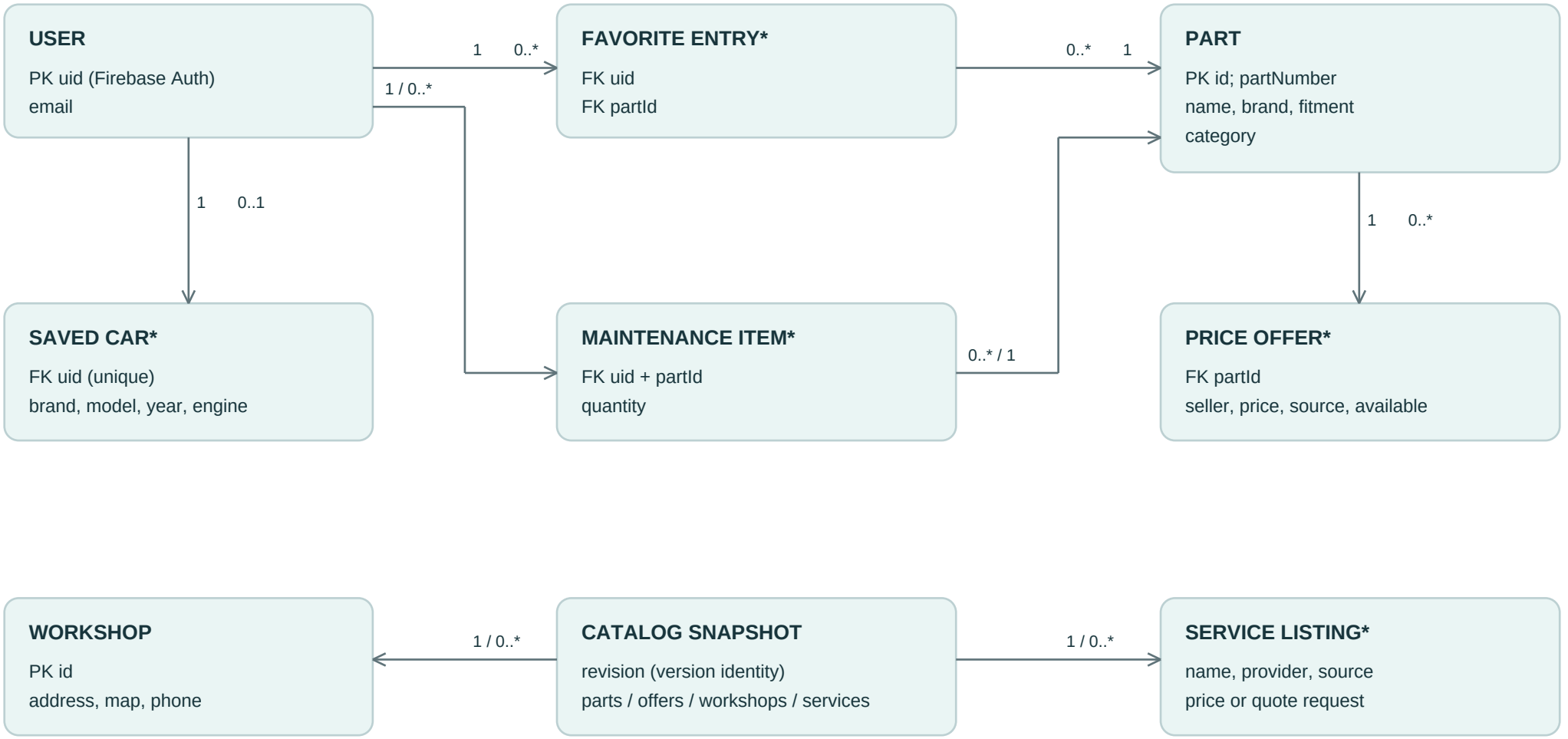


Arrows show containment with multiplicities, not inheritance. GarageData references SparePart IDs through items. SavedCar holds brand, model, year and engine; it represents user input, not verified fitment.

Sources: lib/catalog_data.dart, lib/catalog_snapshot.dart, lib/garage.dart, lib/service_data.dart.

07 ER Diagram

Conceptual entities and cardinalities mapped to the document model.



* Conceptual entity only: favorites, items, car, offers and services are embedded values, not separate collections. There is no verified Workshop-Service relationship; provider names are not foreign keys. PK / FK are conceptual identifiers.

Sources: lib/catalog_snapshot.dart, lib/catalog_data.dart, lib/garage.dart, lib/cloud_stores.dart; conceptual entities marked below.